

From Spectrum Heatmaps to Numeric Time Series: A Pipeline for SpecMon Data Collection, Analysis, and Forecasting

Abstract — This report describes a pipeline that collects spectrum heatmaps from CableLabs SpecMon, converts them to numeric time series, and supports next-day prediction and hourly forecasting. The pipeline has five stages: automated scraping, color-to-value mapping with per-image scale extraction and OCR, transformation to an hourly grid per frequency band, multi-method prediction with confidence and error, and an interactive web viewer with date selection. A time-series notebook adds decomposition and 1-step-ahead forecasting; a validation-tuned blend of last-hour and same-hour-yesterday slightly beats the naive baseline (MAE 2.40 vs 2.42).

1. Introduction

Spectrum monitoring data is often shown as heatmaps (e.g. hour vs frequency, color-coded by utilization). I built a pipeline that turns these into numeric series for analysis and prediction: (1) scrape heatmap images from SpecMon, (2) map colors to values with a calibrated scale and per-image OCR, (3) transform images to hourly tables per frequency band, (4) run next-day prediction with multiple methods and error reporting, and (5) provide a web viewer to compare predicted and actual grids by date. A Jupyter notebook in the `time_series` folder adds decomposition (trend, seasonality, residual) and 1-step-ahead forecasting baselines.

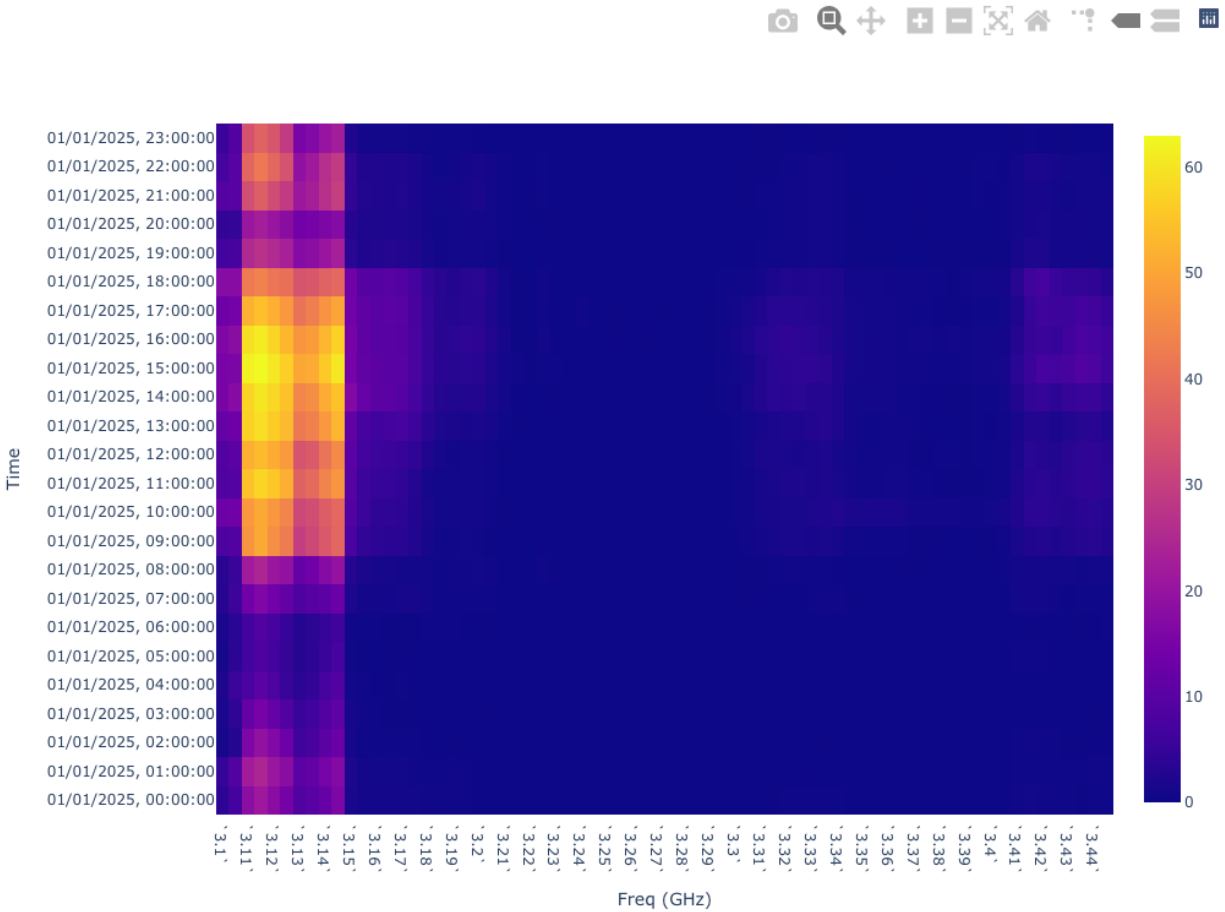
2. Data Collection and Pipeline

2.1 Scraping

Data collection starts at the SpecMon web interface. A browser automation layer performs login using stored credentials, selects the desired location and graph type (e.g. “Hour of Day” heatmap), and configures date range, frequency range (e.g. 3.1–3.45 GHz), and power threshold. For each date in the range, the system captures a screenshot of the heatmap. Raw screenshots are stored; a preprocessing step crops and normalizes the region of interest so that each output image contains a consistent layout: a main graph (hours × frequency bands) and a vertical color scale on the right with a numeric maximum value label. These preprocessed images form the input to

the color and transformation stages. Figure 1 shows an example preprocessed heatmap for one day.

Figure 1. Preprocessed heatmap for one day (hours 0–23, frequency bands 3.1–3.45 GHz). Color encodes value (e.g. utilization); the scale and its maximum are on the right.

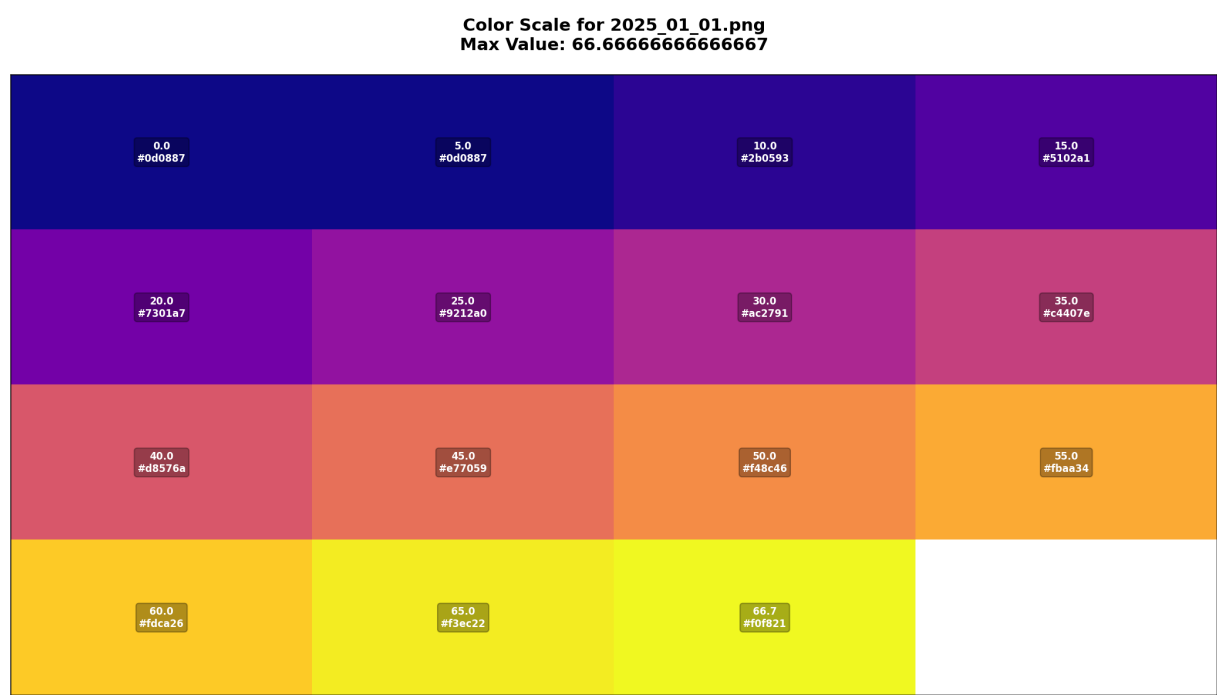


2.2 Color Scale and Validation

The pipeline maps pixel colors to numeric values using a vertical reference scale: dark blue corresponds to zero and yellow to the maximum. The reference scale is stored as a single image and read vertically (top = max, bottom = zero). To improve perceptual consistency, color-to-value conversion is performed in LAB color space: for each pixel, the closest color on the scale (by Euclidean distance in LAB) is found and its position is mapped linearly to a value in $[0, \text{max}]$. When processing heatmap images, the pipeline does not rely solely on the reference scale: for each image, the transformation stage detects the scale region on the right, samples colors along it, and reads the maximum value from the scale label via OCR. The color mapping for that image then uses this per-image scale and max value, so that variations in rendering or labeling across days are accounted for.

Validation: the reference scale can be visualized with value increments, and a *test color scale* step runs on a sample of preprocessed images to validate per-image scale extraction. That step uses the same scale-detection and OCR logic as the transform: find the vertical scale on the right, sample colors along it, read the max value from the label, then build a value-to-color grid (swatches from 0 to max) and save one image per input heatmap. This checks that scale detection and OCR work on real images before running the full transform. Figure 2 shows example test output.

Figure 2. Test color scale output: value-to-color grid from the extracted scale for one preprocessed heatmap (0 to OCR max).



2.3 Image-to-Data Transformation

Each preprocessed image is converted into a numeric table. The transformation stage (1) detects the color scale in the right-hand portion of the image using gradient strength along the vertical direction and fixes a narrow vertical band for the scale; (2) reads the maximum value from the scale label using OCR with multiple regions and preprocessing strategies (contrast, thresholding), falling back to a default if OCR fails; (3) samples colors along the scale bar (top to bottom) and filters out border/background colors; (4) defines a fixed graph region aligned with the heatmap layout (e.g. 24 rows $\times$ 70 columns corresponding to hours 0–23 and frequency bands from 3.1 to 3.45 GHz in 0.005 GHz steps); (5) for each cell, takes the center pixel, maps its color to a value using the per-image scale and max value via the LAB-based color scale; (6) outputs one row per (date, hour) and one column per frequency band into a single combined table (e.g. Parquet). Optional debug outputs can save images showing the

detected scale region and the grid of sampled cell centers. Figures 3 and 4 show examples of scale-region and dataframe-sampling debug visualizations.

Figure 3. Detected scale region on a preprocessed heatmap (debug output).

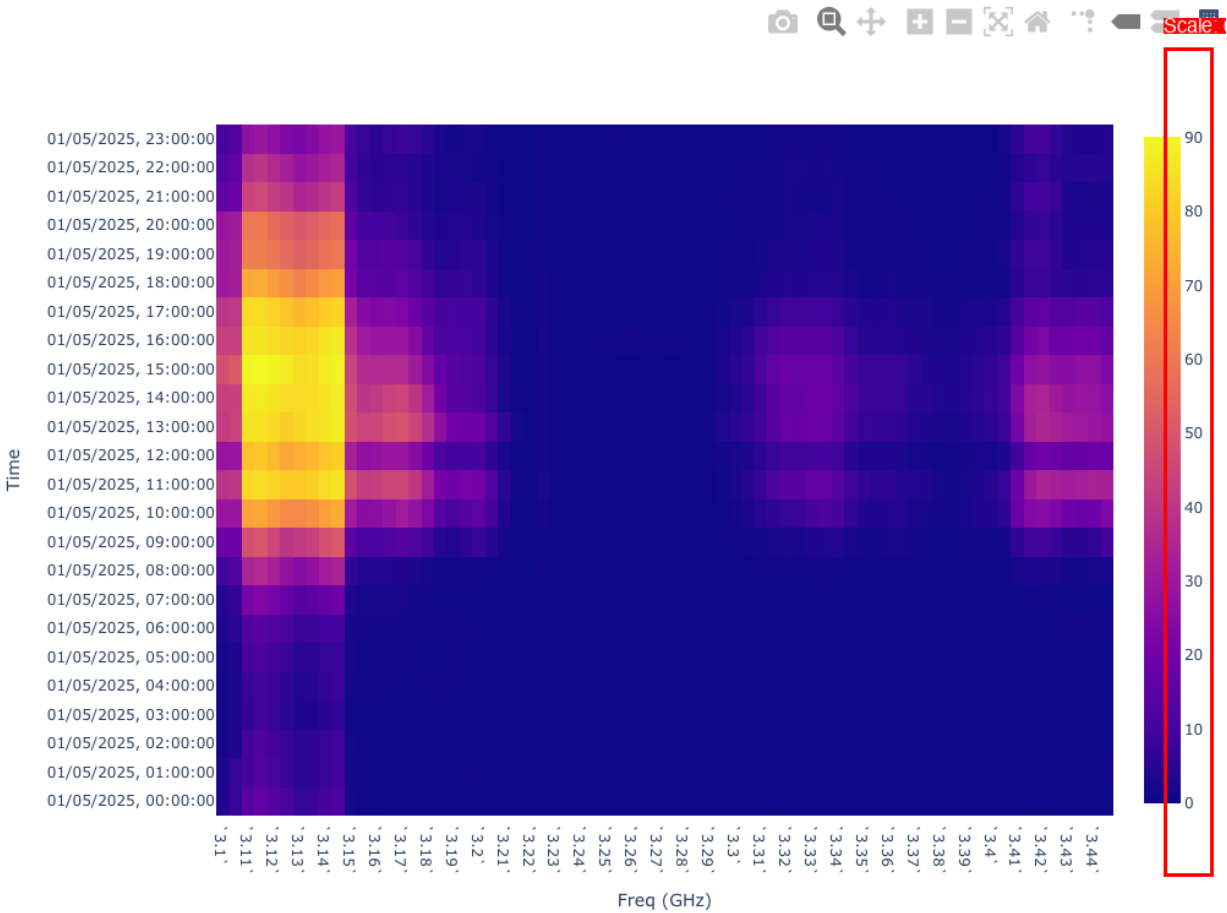
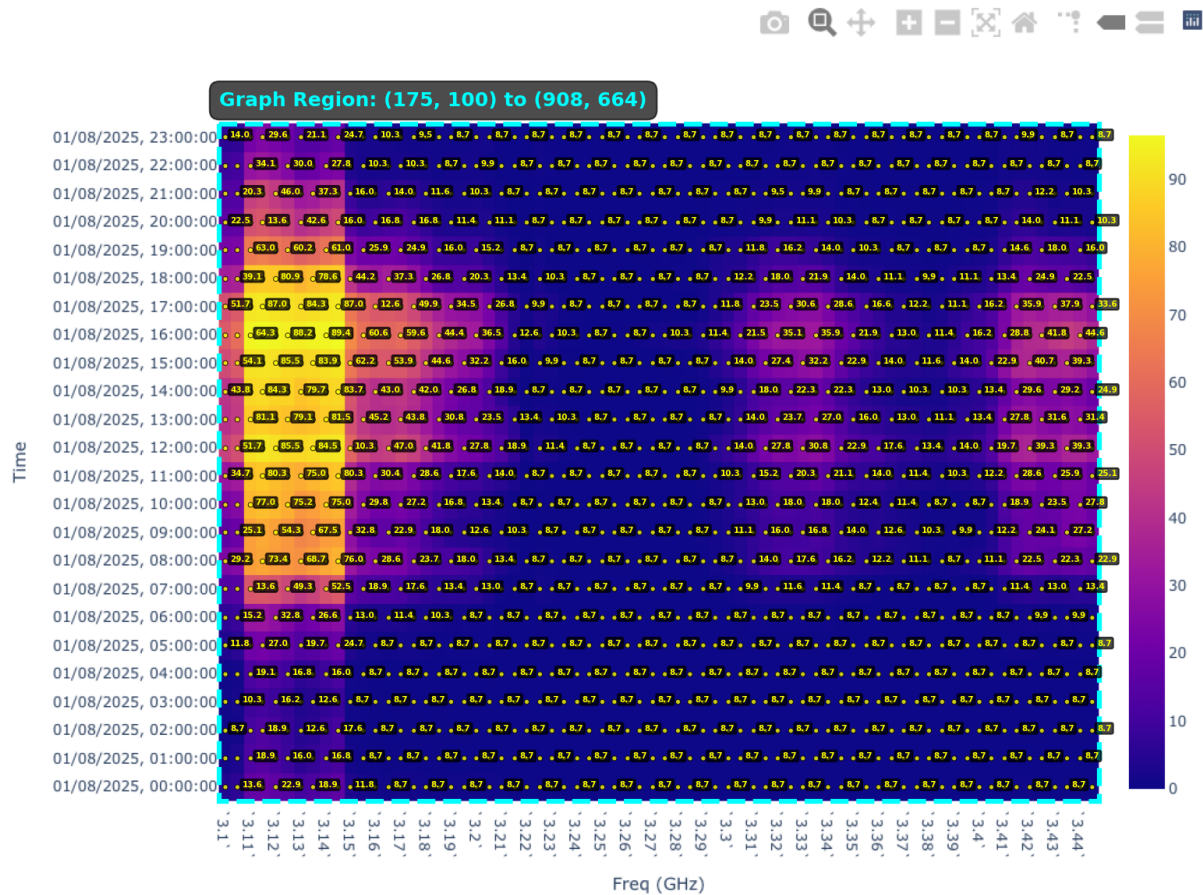


Figure 4. Grid of sampling points over the graph (debug output).

Dataframe Extraction: 2025_01_08.png



3. Analysis and Prediction

The transformed table (one row per date-hour, columns for each frequency band) is reshaped into a long form with (date, hour, frequency_band, value). The analysis stage produces next-day predictions for every (hour, frequency_band) cell, along with confidence scores and errors when actuals are available. When creating this model, I believed that online learning could be used effectively with simple models such as exponential smoothing, linear regression, or moving average; the implementation reflects that choice.

3.1 Prediction Methods

Four prediction methods are implemented and can be selected per cell by an online learner (see below).

- **Exponential smoothing:** The next value is a convex combination of the previous value and the new observation; the smoothing parameter is set close to 1 so the forecast is near the last observation (persistence-like).

- **Moving average:** The prediction is the mean of the last few observations (e.g. last two hours) for that (hour, frequency_band) across recent days.
- **Linear regression:** A linear trend is fit over the lookback window for that cell; the prediction is the extrapolated value at the next time step.
- **Peak detection:** When recent history shows high volatility and a clear peak (e.g. value above a multiple of the mean and above a fraction of the recent maximum), the prediction is a damped function of that peak. This targets cells with sporadic high utilization.

Each method uses a configurable lookback (number of past days). In effect, the visualized predictions are seasonal exponential smoothing or linear models over the past six days (same hour, same frequency band across days). Error is calculated per day (comparing predicted vs actual for the full day's grid) rather than per hour; only after framing the problem as time series did I recognize that this is what I had implemented.

3.2 Confidence and Error

A confidence score in $[0, 1]$ is computed for each prediction. The score combines data availability, variation in the history, consistency with the chosen method, and accuracy history (recent normalized errors). Weights and decay are configurable. Error is defined as actual minus predicted. Normalized error is used to update accuracy history. Aggregate statistics over the error map (mean absolute error, max, min, fraction of cells with error below a small threshold) are computed for reporting.

3.3 Online Learning

The pipeline maintains persistent state per (hour, frequency_band) cell: for each prediction method, it stores a running list of accuracies (derived from normalized error when actuals become available). When making a prediction, the system selects the method with the best recent accuracy for that cell, with a small probability of choosing a random method for exploration. Lookback length can also be learned over time from daily accuracy. State is saved to disk so that each run continues from the previous one.

3.4 Visualization

After the analysis step completes, an interactive visualization is launched and opens in a web browser. It loads the prediction table and provides a **date selector** so that the user can view the full grid (24×70: hours × frequency bands) for **any day**. View modes include: **Predicted** (heatmap of predicted values), **Confidence** (heatmap of

confidence scores), **Actual** (heatmap of observed values when available), and **Error** (signed error heatmap). This gives better insight than static images alone: predicted and actual values can be compared side by side for any chosen date. Summary statistics (mean confidence, mean/max/min error, fraction of near-zero errors) are shown. Figures 6–8 show the prediction viewer with date selector and example predicted and actual grids.

The analysis stage can also write static heatmap images for selected dates. Figure 5 shows an example prediction heatmap for one day.

Figure 5. Prediction heatmap for one day (analysis output).

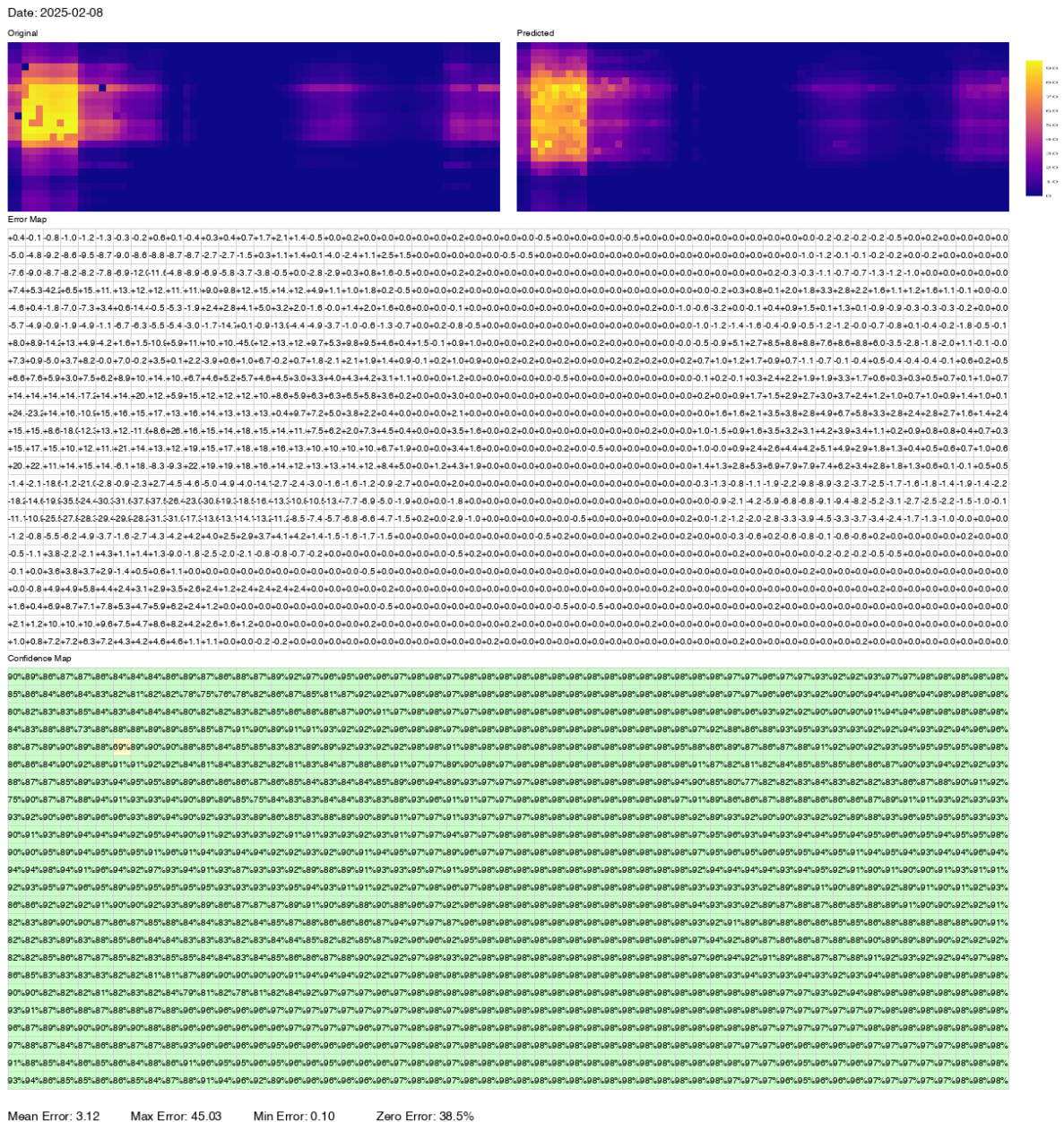


Figure 6. Prediction viewer: date selector and view options (runs in browser after analysis).

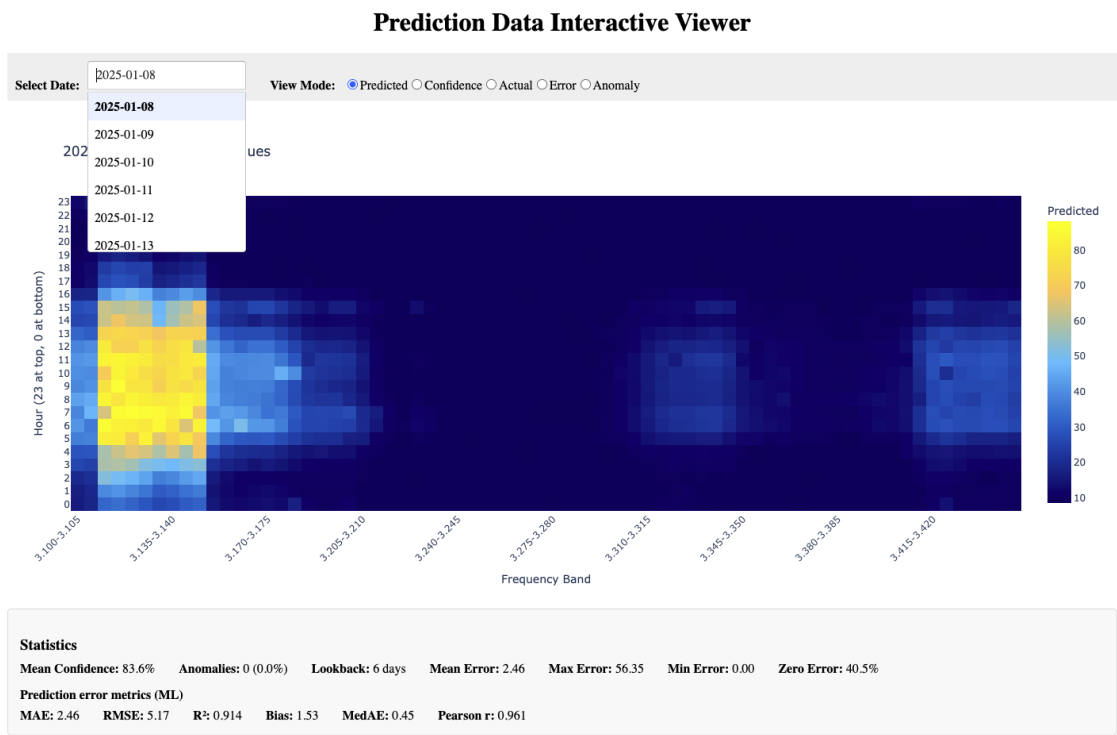


Figure 7. Predicted values for a selected day (grid: hours × frequency bands).

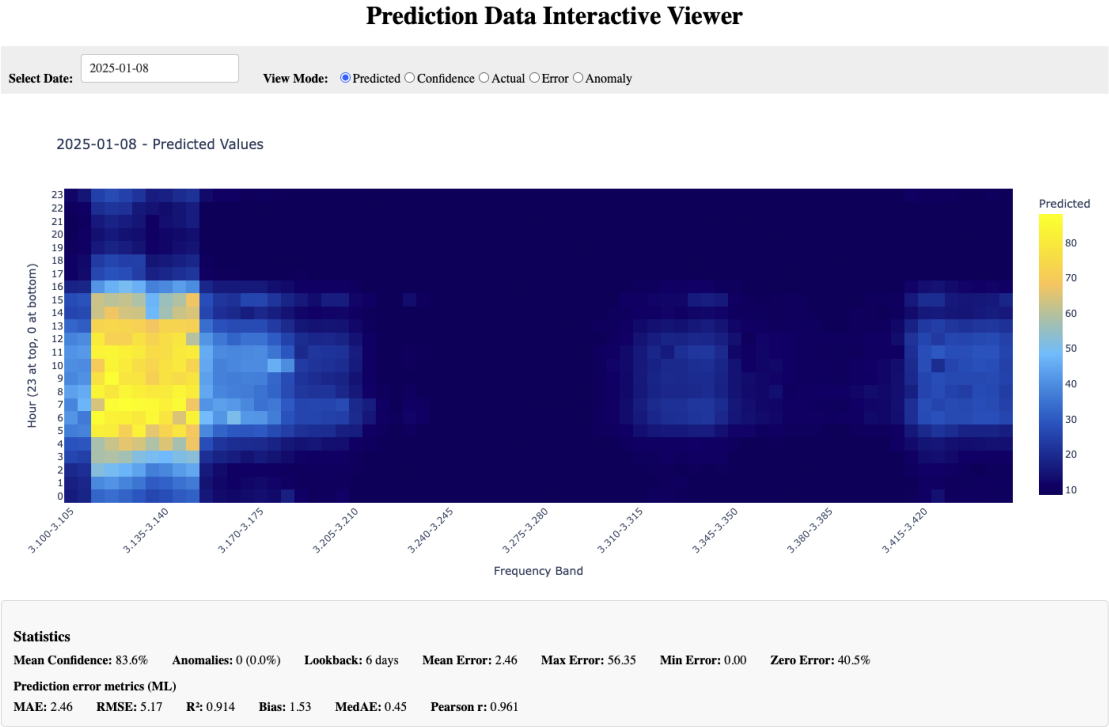
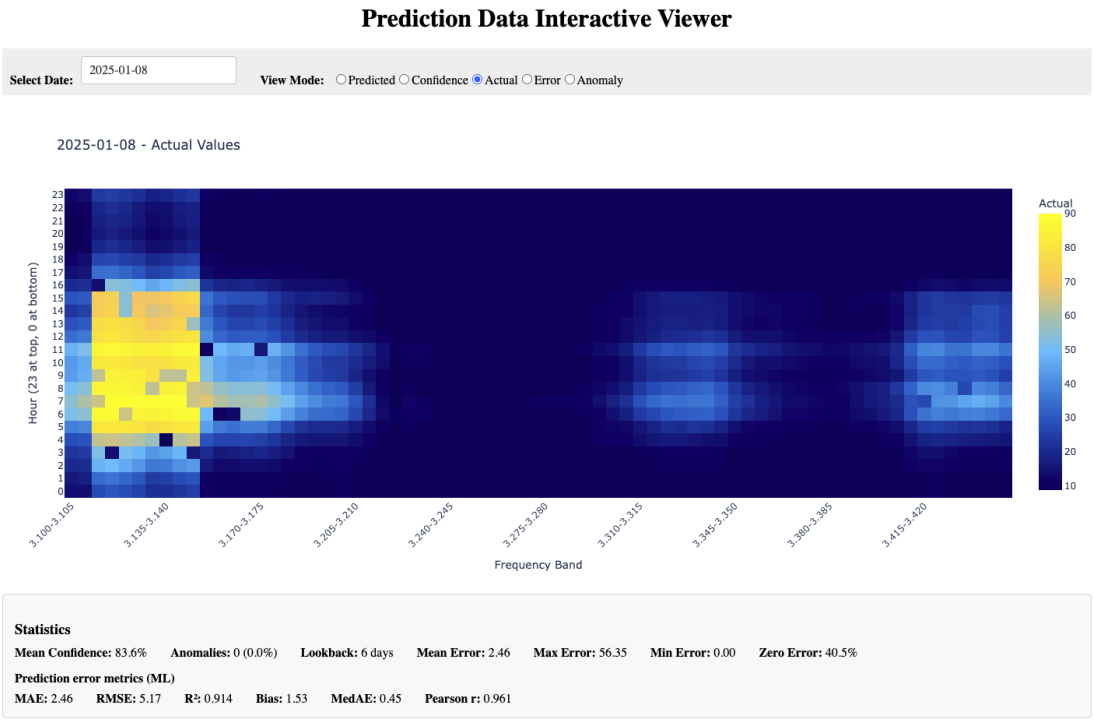


Figure 8. Actual values for the same day (grid: hours × frequency bands).

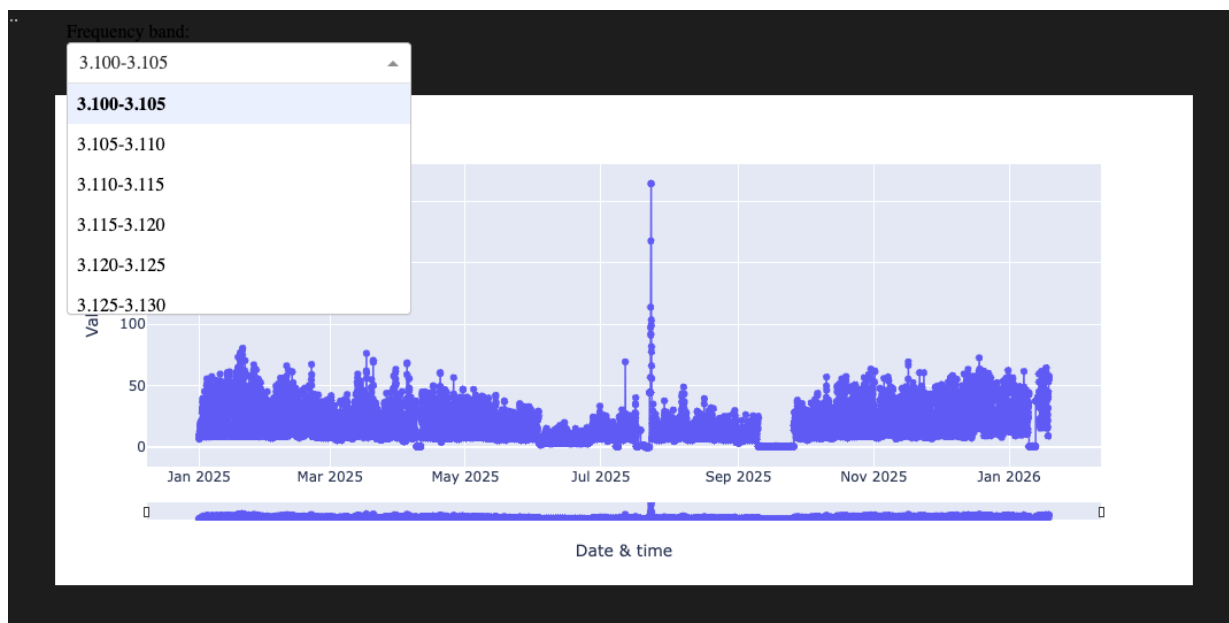


4. Time Series Exploration

4.1 Motivation

The transformed and prediction data are naturally indexed by (date, hour, frequency_band). For each frequency band, the sequence of hourly values forms a time series. Only after learning about time series did I recognize that the analysis-stage model (seasonal exponential smoothing / linear over the past six days, with error per day) was a particular time-series formulation. Once I realized the data was time series, I created a Jupyter notebook in the time_series folder. That notebook loads the same transformed table, defines a fixed test window (e.g. the last year of hours), and evaluates 1-step-ahead forecasting models in a unified way (MAE, RMSE, MASE). The goal is to understand structure (trend, seasonality) and to see whether any model can systematically outperform a naive (last observation) baseline. The notebook provides a dropdown to select a frequency band and view the raw hourly series for that band. Figure 9 shows the raw time series with the frequency-range selector.

Figure 9. Raw time series for a selected frequency band (notebook UI with dropdown).



4.2 Decomposition

I apply additive decomposition to each frequency band's series: **raw** = **trend** + **seasonal** + **residual**. The three components are defined as follows.

- **Trend** is the slow-moving, smooth level of the series over time. It captures long-term growth, decline, or drift (e.g. a gradual rise in utilization over weeks) and is typically estimated by a centered moving average over the seasonal period. It has no fixed repeating pattern.
- **Seasonality** is the repeating pattern at a fixed period (e.g. 24 hours for daily, 168 hours for weekly). It captures regular cycles—for example, higher values at certain hours each day or certain days each week—and is computed by extracting the periodic component after removing the trend.
- **Residual** is what remains after subtracting trend and seasonal from the raw series. It represents irregular, unpredictable variation: noise, one-off spikes, or structure that is not captured by trend or the chosen seasonal period. A well-fitting decomposition leaves residuals that look roughly random.

The seasonal component is computed with a fixed period; the notebook supports daily (24-hour) and weekly (168-hour) periods. Trend is obtained by a centered moving average over the period length, with extrapolation at the ends so that the residual is defined everywhere. The decomposition is visualized in four panels (raw, trend, seasonal, residual) so that daily vs weekly seasonality can be compared. This confirms a strong daily pattern and suggests that a weekly component may also be present in some bands. Figures 10 and 11 show the full decomposition view and a zoomed-in segment.

Figure 10. Decomposition into raw, trend, seasonal, and residual (four panels).

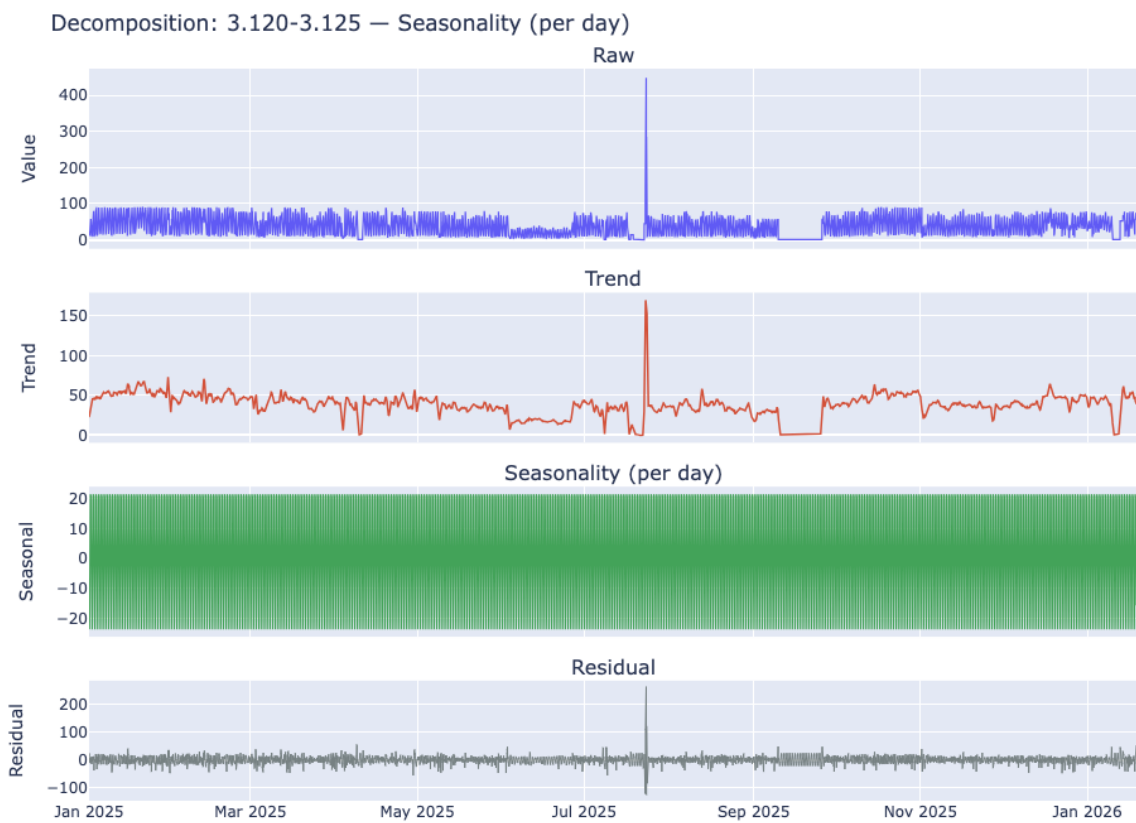
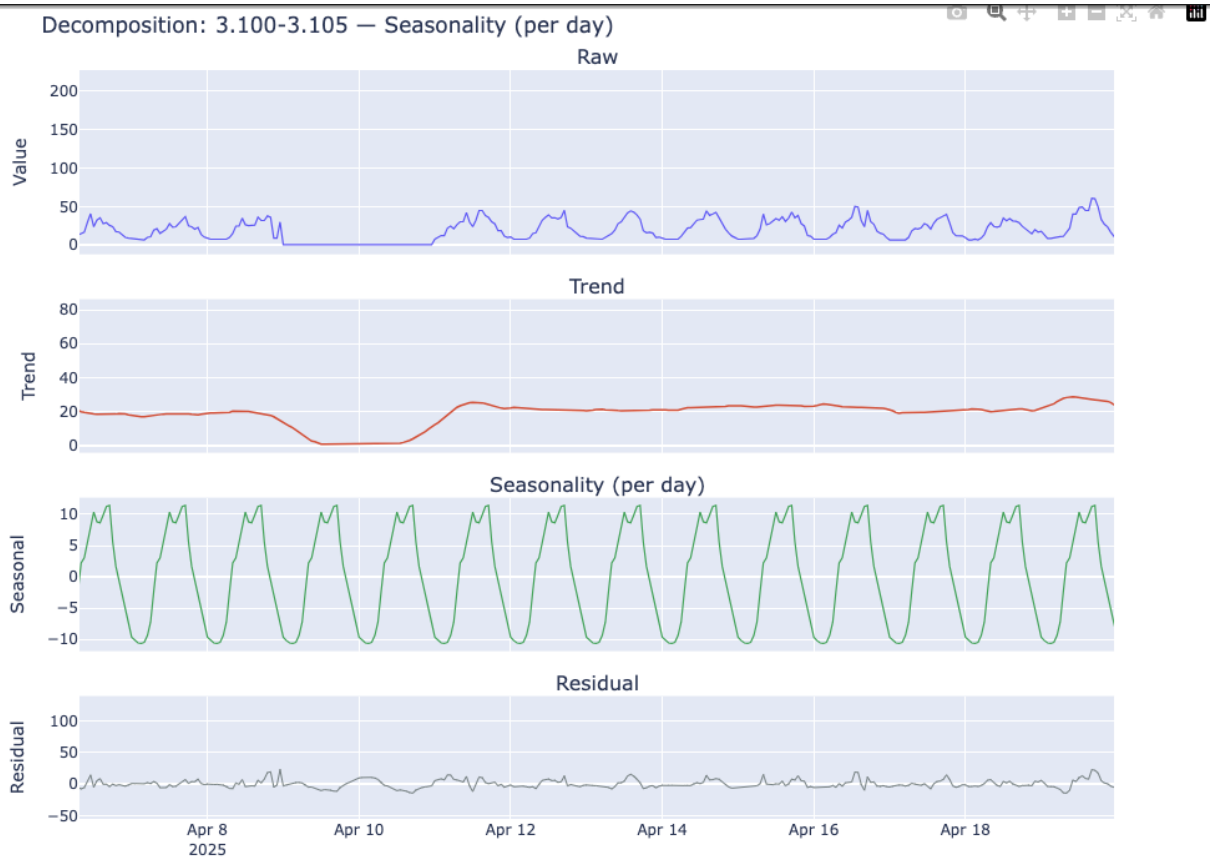


Figure 11. Decomposition zoomed in to show detail.



4.3 Forecasting Models Evaluated

All models are evaluated on the same test set (last 365×24 hours) with 1-step-ahead predictions. Metrics are MAE, RMSE, and MASE (scaled by the mean absolute first difference of the actuals on the test set, so that $\text{MASE} = 1$ for the naive forecast). The following were implemented and evaluated.

Baselines and simple methods

- **Naive:** Prediction = previous hour's value. Serves as the main baseline ($\text{MASE} = 1$ by construction).
- **Moving average:** Prediction = mean of the last k hours (e.g. $k = 2$).
- **Exponential smoothing:** Prediction = $\alpha \times \text{previous value} + (1 - \alpha) \times \text{previous forecast}$; α close to 1 (e.g. 0.98).
- **Seasonal naive:** Prediction = value at the same hour one day ago (lag 24).
- **Median ensemble:** Prediction = median of naive and seasonal naive.

Blends and linear combinations

- **Hybrid (fixed blend):** 70% last hour + 30% same hour yesterday (fixed weights).
- **OLS blend:** For each band, fit $\hat{y} = a \cdot y(t-1) + b \cdot y(t-24) + c$ on the training period by least squares; predict on the test set.
- **Tuned blend:** For each band, choose α in $[0.97, 1]$ to minimize MAE on a short validation window (e.g. last 14 days before the test set); prediction = $\alpha \cdot y(t-1) + (1-\alpha) \cdot y(t-24)$. This keeps the forecast naive-heavy while allowing a small daily-seasonal correction.
- **Tuned blend with longer validation:** Same as tuned blend but with a 28-day validation window.
- **Three-way blend:** For each band, choose (α, β, γ) with $\alpha + \beta + \gamma = 1$ to minimize validation MAE; prediction = $\alpha \cdot y(t-1) + \beta \cdot y(t-24) + \gamma \cdot y(t-168)$. This adds a weekly (168-hour) seasonal component.
- **Ridge with lags:** For each band, fit a Ridge regression of $y(t)$ on a constant, $y(t-1)$, $y(t-24)$, and $y(t-168)$; predict on the test set.

Statistical and ML models

- **ARIMA:** Rolling 1-step-ahead ARIMA(1,0,1) per band on a subset of bands (e.g. last 3 days of test for computational reasons).
- **Chronos:** A pretrained time-series foundation model (Chronos-2) used in a rolling-window setting (e.g. 72-hour horizon per window) for comparison; evaluation is aligned where possible with the 1-step setup.

- **Seasonal weighted average:** For each hour, prediction = weighted average of the same hour on the previous N days (e.g. $N = 7$), with higher weight on more recent days.
- **Seasonal exponential smoothing:** Exponential smoothing applied to the seasonal (same-hour-yesterday) series.

How each experiment was tuned. All models use the same test set (last 365×24 hours); no test data is used for tuning. **Naive** and **seasonal naive** have no hyperparameters. **Moving average** uses a fixed window (e.g. $k = 2$ hours).

Exponential smoothing uses a fixed α close to 1 (e.g. 0.98) so the forecast is persistence-like. **Hybrid** uses fixed weights (70% last hour, 30% same hour yesterday). **OLS blend** is fit by least squares on the training (pre-test) period only; no validation tuning. **Median ensemble** is the median of naive and seasonal naive; no tuning. **Tuned blend** is the only model that uses a validation set: for each frequency band, $\alpha \in [0.97, 1.0]$ is chosen to minimize MAE on a short validation window (e.g. 14 days immediately before the test set); the same validation period is used for all bands. **Seasonal weighted average** uses a fixed N (e.g. 7 days) with higher weight on more recent days. **Seasonal exponential smoothing** uses the past 6 days at the same hour with smoothing that weights recent days more. **ARIMA** uses a fixed order (e.g. (1,0,1)); **Chronos-2** is pretrained and used with a fixed rolling-window horizon (e.g. 72 hours). No model sees the test set until the final evaluation.

4.4 Results (MAE, RMSE, MASE)

All metrics are on the same test set (last 365×24 hours, 1-step-ahead). **MAE** (mean absolute error) is the average of $|\text{actual} - \text{predicted}|$; **RMSE** (root mean squared error) is the square root of the average of $(\text{actual} - \text{predicted})^2$; **MASE** (mean absolute scaled error) is MAE of the model divided by the MAE of the naive 1-step forecast on the test set, so the naive baseline has $\text{MASE} = 1$ by construction—values below 1 indicate better than naive, above 1 indicate worse.

Model	MAE	RMSE	MASE
Naive	2.4199	6.2706	1.0000
Tuned blend	2.4041	6.1691	0.9935
Exponential smoothing	2.4302	6.2546	1.0043
Hybrid (70/30)	2.5157	6.0237	1.0396
OLS blend	2.7532	6.0416	1.1450

Model	MAE	RMSE	MASE
Median ensemble	2.8368	6.8291	1.1723
Moving average	2.8825	6.6104	1.1912
ARIMA	6.5317	9.6009	1.1288
Chronos-2	3.6528	8.8533	1.5249
Seasonal weighted avg	4.0999	9.1835	1.6943
Seasonal exponential smoothing	4.0173	9.3046	1.6602
Seasonal naive	4.2875	10.7625	1.7718

The best performer is the **tuned blend** (MAE 2.4041, MASE 0.9935), slightly better than naive (MAE 2.4199, MASE 1.0000). Exponential smoothing is very close to naive. The hybrid (70% last hour, 30% same hour yesterday) and OLS blend are next; moving average and pure seasonal methods (seasonal naive, seasonal weighted average, seasonal exponential smoothing) do worse. ARIMA and Chronos-2, in these configurations, do not beat the simple tuned blend on this 1-step metric.

Figure 12. Example error (e.g. MAE or MASE) per model, illustrating relative performance across forecasting methods.

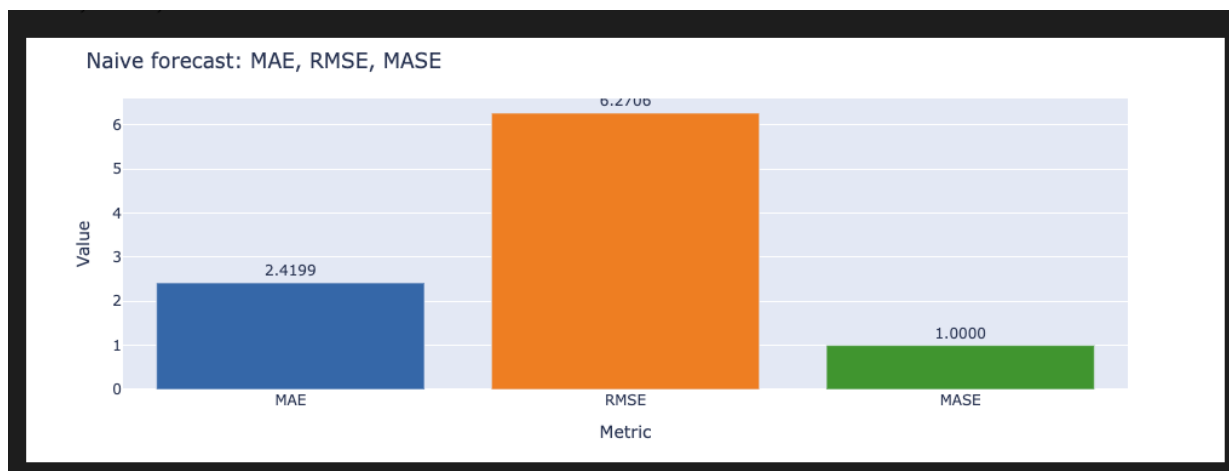
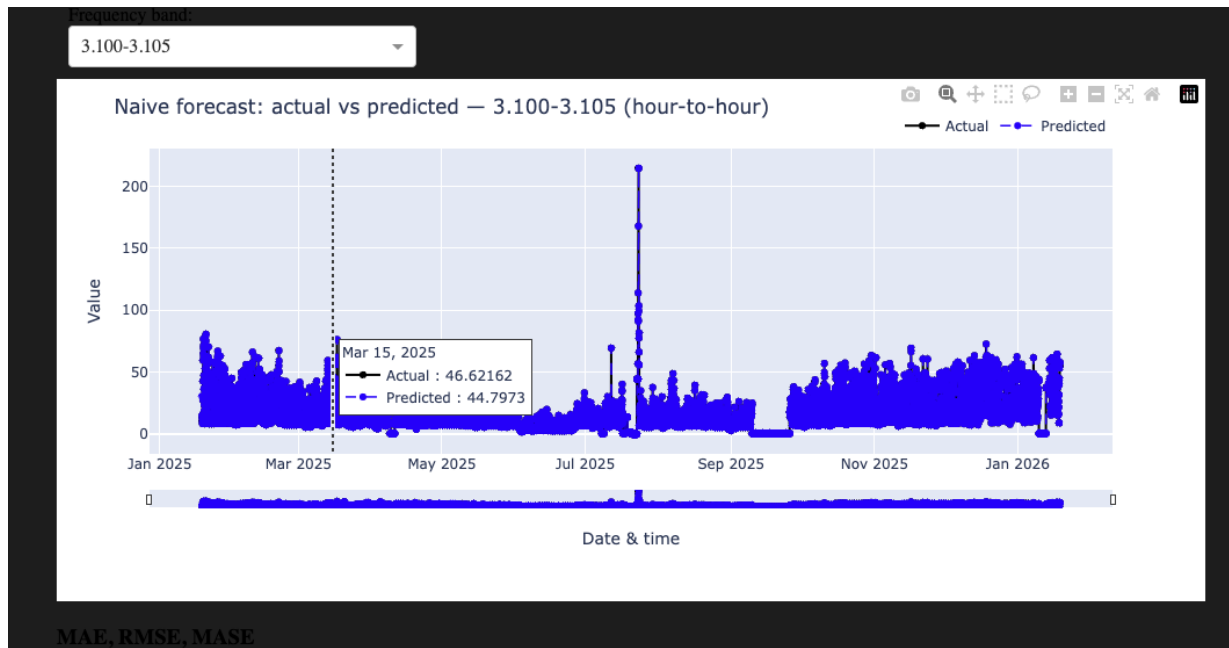


Figure 13. Example time series plot: actual vs predicted for each model (or a selected subset), showing how predictions track the actual series over time.



4.5 Diebold–Mariano Test

I use the **Diebold–Mariano (DM) test** to assess whether one model has significantly different MAE than another. For two error sequences I form $d_t = |e_{1,t}| - |e_{2,t}|$ and compute $DM = \bar{d}/(\sigma_d/\sqrt{n})$; under equal accuracy this is approximately standard normal. I report a two-sided p -value; $p < 0.05$ means I reject equal accuracy. $DM > 0 \Rightarrow$ first model worse (second has lower MAE); $DM < 0 \Rightarrow$ first model better. In the notebook I merge predictions on (datetime, frequency_band), run pairwise DM for selected model pairs, and report n , DM, p , and which model has lower MAE. In practice the DM test did not add much: pairwise p -values were effectively zero (very high confidence that differences exist), so all comparisons were “significant” and the test did not discriminate. **MASE** was the final determiner of the winner (tuned blend with $MASE < 1$).

4.6 Winner (Tuned Blend) Visualization

The winning model (tuned blend) can be visualized via the *analyze-winner* and *visualize-winner* tasks, which produce predicted and actual heatmaps for any selected day. Figures 14 and 15 show example winner predicted and actual grids for the same day. **Caveat:** the tuned blend is effectively very close to the naive method: $\alpha \in [0.97, 1]$ means the forecast is dominated by the previous hour (last observation), with only a small weight on the same hour yesterday, so behavior and heatmap appearance are similar to naive. Note that the predicted heatmap often appears **shifted downward** relative to the actual: because the previous hour’s weight is high, each row of the prediction is strongly influenced by the row above (last hour), so patterns in the actual data tend to appear shifted down by one hour in the prediction.

Figure 14. Example winner predicted (Tuned blend) for one day (grid: hours × frequency bands).

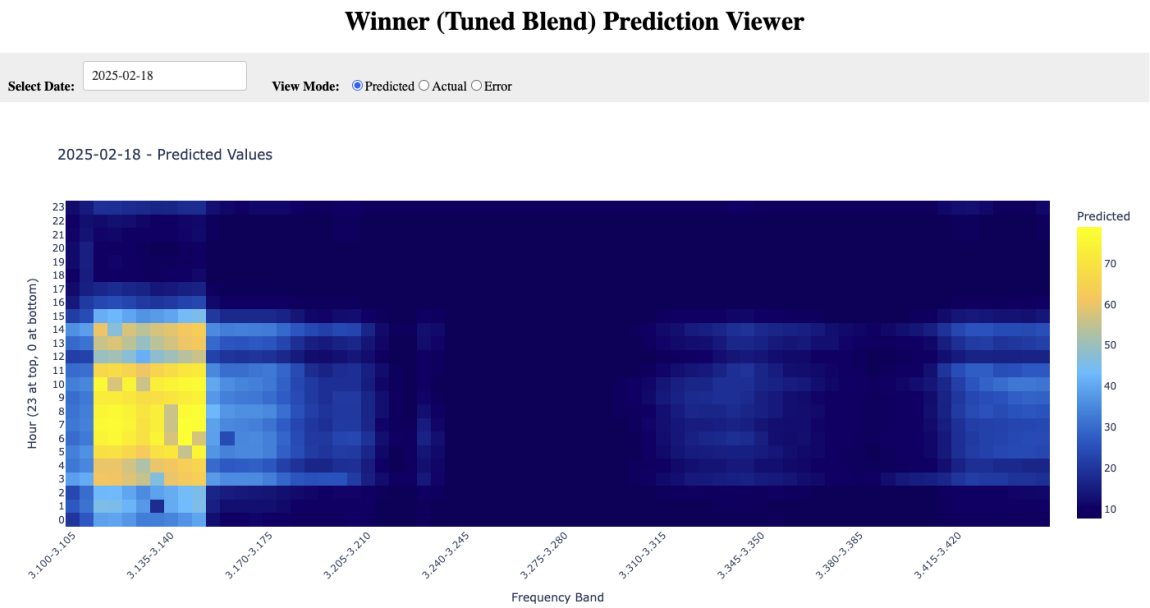


Figure 15. Example winner actual for the same day (grid: hours × frequency bands).

